

Nama : Yudhistira Putra Hartanto

Kelas/Absen : 1G/29

NIM : 254107020083

6.2. Searching/ Pencarian Menggunakan Algoritma Sequential Search

- Mahasiswa29

```
package jobsheet6;
```

```
public class Mahasiswa29 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    Mahasiswa29(){
```

```
}
```

```
    Mahasiswa29(String nm, String name, String kls, double ip){
```

```
        nim = nm;
```

```
        nama = name;
```

```
        kelas = kls;
```

```
        ipk = ip;
```

```
    }
```

```
    void tampil(){
```

```
        System.out.println("NIM \t: " + nim);
```

```
        System.out.println("NAMA \t: " + nama);
```

```
        System.out.println("KELAS \t: " + kelas);
```

```
        System.out.println("IPK \t: " + ipk);
```

```
    }
```

```
}
```

- MahasiswaBerprestasi29

```
package jobsheet6;
```

```
public class MahasiswaBerprestasi29 {
```

```
    Mahasiswa29 [] listmhs = new Mahasiswa29 [5];
```

```
    int idx;
```

```
    void tambah(Mahasiswa29 mhs){
```

```
        if (idx < listmhs.length) {
```

```
            listmhs[idx] = mhs;
```

```
            idx++;
```

```
        } else {
```

```
            System.out.println("Data sudah penuh!");
```

```
        }
```

```
    }
```

```

void tampil () {
    for (int i = 0; i < idx; i++) {
        listmhs[i].tampil();
        System.out.println("-----");
    }
}

int sequentialSearch(double cari){
    int posisi = -1;
    for (int j=0; j<listmhs.length; j++){
        if (listmhs[j].ipk == cari){
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos){
    if (pos != -1){
        System.out.println("data mahasiswa dengan IPK : " + x + " ditemukan pada indeks " + pos);
    }
    else {
        System.out.println("data " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos){
    if (pos != -1){
        System.out.println("nim\t : " + listmhs[pos].nim);
        System.out.println("nama\t : " + listmhs[pos].nama);
        System.out.println("kelas\t : " + listmhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    }
    else {
        System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
    }
}
}

```

- **MahasiswaDemo**

```
package jobsheet6;
```

```
import java.util.Scanner;
```

```

public class MahasiswaDemo29 {
    public static void main(String[] args) {
        MahasiswaBerprestasi29 list = new MahasiswaBerprestasi29();
        Scanner sc = new Scanner(System.in);
        int jumMhs = 5;
    }
}

```

```

for (int i = 0; i < jumMhs; i++) {
    System.out.println("Masukkan Data Mahasiswa ke-" + (i + 1));
    System.out.print("NIM : ");
    String nim = sc.nextLine();
    System.out.print("Nama : ");
    String nama = sc.nextLine();
    System.out.print("Kelas : ");
    String kelas = sc.nextLine();
    System.out.print("IPK : ");
    String ipk = sc.nextLine();
    Double ipk = Double.parseDouble(ipk);
    System.out.println("---");
    list.tambah(new Mahasiswa29(nim, nama, kelas, ipk));
}

list.tampil();
System.out.println("---");
System.out.println("Pencarian data");
System.out.println("---");
System.out.println("Masukkan IPK mahasiswa yang dicari: ");
System.out.print("IPK: ");
double cari = sc.nextDouble();

System.out.println("Menggunakan sequential searching");
double posisi = list.sequentialSearch(cari);
int pss = (int) posisi;
list.tampilPosisi(cari, pss);
list.tampilDataSearch(cari, pss);
}
}

```

- Hasil

Masukkan Data Mahasiswa ke-1

NIM : 111

Nama : adi

Kelas : 2

IPK : 3.6

Masukkan Data Mahasiswa ke-2

NIM : 222

Nama : tio

Kelas : 2

IPK : 3.8

Masukkan Data Mahasiswa ke-3

NIM : 333

Nama : ila

Kelas : 2

IPK : 3.0

Masukkan Data Mahasiswa ke-4

NIM : 444

Nama : lia
Kelas : 2
IPK : 3.5

Masukkan Data Mahasiswa ke-5

NIM : 555
Nama : fia
Kelas : 2
IPK : 3.3

NIM : 111
NAMA : adi
KELAS : 2
IPK : 3.6

NIM : 222
NAMA : tio
KELAS : 2
IPK : 3.8

NIM : 333
NAMA : ila
KELAS : 2
IPK : 3.0

NIM : 444
NAMA : lia
KELAS : 2
IPK : 3.5

NIM : 555
NAMA : fia
KELAS : 2
IPK : 3.3

Pencarian data

Masukkan IPK mahasiswa yang dicari:

IPK: 3,5

Menggunakan sequential searching

data mahasiswa dengan IPK : 3.5 ditemukan pada indeks 3

nim : 444
nama : lia
kelas : 2
ipk : 3.5

6.2.3. Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!
2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){  
    posisi=j;  
    break;  
}
```

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.
5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

Jawaban :

1. Perbedaannya adalah method tampilPosisi hanya menampilkan indeks/posisi ditemukannya data dn nilai IPK yang dicari, sedangkan method tampilDataSearch menampilkan seluruh data mahasiswa (NIM,nama,kelas,IPK) pada indeks tersebut
2. Break pada kode program tersebut berfungsi untk menghentikan perulangan segera setelah data yang dicari ditemukan.
3. Variabel pos disini itu berfungsi untuk menyimpan indeks lokasi data yang ditemukan di dalam array.
4. Akan menampilkan data pertama yang ditemukan karena setelah menemukan kecocokan data pertama yang sama maka break akan langsung menghentikan perulangan
5. Program akan tetap mencari hingga akhir array meskipun data yang cocok/sama dengan yang dicari telah ditemukan

6.3. Searching/ Pencarian Menggunakan Algoritma Binary Search

- Mahasiswa29

```
package jobsheet6;
```

```
public class Mahasiswa29 {
```

```
    String nim;  
    String nama;  
    String kelas;  
    double ipk;
```

```
    Mahasiswa29(){
```

```
}
```

```
    Mahasiswa29(String nm, String name, String kls, double ip){
```

```
        nim = nm;  
        nama = name;  
        kelas = kls;  
        ipk = ip;  
    }
```

```
    void tampil(){
```

```
        System.out.println("NIM \t: " + nim);  
        System.out.println("NAMA \t: " + nama);  
        System.out.println("KELAS \t: " + kelas);  
        System.out.println("IPK \t: " + ipk);  
    }
```

```
}
```

- MahasiswaBerprestasi29

```
package jobsheet6;
```

```
public class MahasiswaBerprestasi29 {
```

```
    Mahasiswa29 [] listmhs = new Mahasiswa29 [5];  
    int idx;
```

```
    void tambah(Mahasiswa29 mhs){
```

```
        if (idx < listmhs.length) {  
            listmhs[idx] = mhs;  
            idx++;  
        } else {  
            System.out.println("Data sudah penuh!");  
        }  
    }
```

```
    void tampil () {
```

```
        for (int i = 0; i < idx; i++) {  
            listmhs[i].tampil();  
            System.out.println("-----");  
        }  
    }
```

```

int sequentialSearch(double cari){
    int posisi = -1;
    for (int j=0; j<listmhs.length; j++){
        if (listmhs[j].ipk == cari){
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos){
    if (pos != -1){
        System.out.println("data mahasiswa dengan IPK : " + x + " ditemukan pada indeks " + pos);
    }
    else {
        System.out.println("data " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos){
    if (pos != -1){
        System.out.println("nim\t : " + listmhs[pos].nim);
        System.out.println("nama\t : " + listmhs[pos].nama);
        System.out.println("kelas\t : " + listmhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    }
    else {
        System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
    }
}

int findBinarySearch(double cari, int left, int right){
    int mid;
    if (right >= left){
        mid = (left + right) / 2;
        if (cari == listmhs[mid].ipk){
            return (mid);
        }
        else if (listmhs[mid].ipk > cari){
            return findBinarySearch(cari, left, mid - 1);
        }
        else{
            return findBinarySearch(cari, mid + 1, right);
        }
    }
    return -1;
}

```

- MahasiswaDemo29

```

package jobsheet6;

import java.util.Scanner;

public class MahasiswaDemo29 {
    public static void main(String[] args) {
        MahasiswaBerprestasi29 list = new MahasiswaBerprestasi29();
        Scanner sc = new Scanner(System.in);
        int jumMhs = 5;

        for (int i = 0; i < jumMhs; i++) {
            System.out.println("Masukkan Data Mahasiswa ke-" + (i + 1));
            System.out.print("NIM \t: ");
            String nim = sc.nextLine();
            System.out.print("Nama \t: ");
            String nama = sc.nextLine();
            System.out.print("Kelas \t: ");
            String kelas = sc.nextLine();
            System.out.print("IPK \t: ");
            String ip = sc.nextLine();
            Double ipk = Double.parseDouble(ip);
            System.out.println("---");
            list.tambah(new Mahasiswa29(nim, nama, kelas, ipk));
        }

        list.tampil();
        System.out.println("---");
        System.out.println("Pencarian data");
        System.out.println("---");
        System.out.println("Masukkan IPK mahasiswa yang dicari: ");
        System.out.print("IPK: ");
        double cari = sc.nextDouble();

        System.out.println("Menggunakan sequential searching");
        double posisi = list.sequentialSearch(cari);
        int pss = (int) posisi;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);

        // melakukan pencarian data Binary
        System.out.println("");
        System.out.println("Pencarian data");
        System.out.println("---");
        System.out.println("masukkan ipk mahasiswa yang dicari: ");
        System.out.print("IPK: ");
        double caribinary = sc.nextDouble();
        System.out.println("---");
        System.out.println("menggunakan binary search");
        System.out.println("---");
        double posisi2 = list.findBinarySearch(caribinary, 0, jumMhs - 1);
        int pss2 = (int) posisi2;
        list.tampilPosisi(caribinary, pss2);
        list.tampilDataSearch(caribinary, pss2);
    }
}

```

```
}  
}
```

- Hasil

Masukkan Data Mahasiswa ke-1

NIM : 111

Nama : adi

Kelas : 2

IPK : 3.1

Masukkan Data Mahasiswa ke-2

NIM : 222

Nama : ila

Kelas : 2

IPK : 3.2

Masukkan Data Mahasiswa ke-3

NIM : 333

Nama : lia

Kelas : 2

IPK : 3.3

Masukkan Data Mahasiswa ke-4

NIM : 444

Nama : susi

Kelas : 2

IPK : 3.5

Masukkan Data Mahasiswa ke-5

NIM : 555

Nama : anita

Kelas : 2

IPK : 3.7

NIM : 111

NAMA : adi

KELAS : 2

IPK : 3.1

NIM : 222

NAMA : ila

KELAS : 2

IPK : 3.2

NIM : 333

NAMA : lia

KELAS : 2

IPK : 3.3

NIM : 444

NAMA : susi

KELAS : 2

IPK : 3.5

```
-----  
NIM    : 555  
NAMA   : anita  
KELAS  : 2  
IPK    : 3.7  
-----
```

```
---  
Pencarian data  
---
```

```
Masukkan IPK mahasiswa yang dicari:  
IPK: 3,7  
Menggunakan sequential searching  
data mahasiswa dengan IPK : 3.7 ditemukan pada indeks 4  
nim     : 555  
nama    : anita  
kelas   : 2  
ipk     : 3.7
```

```
Pencarian data
```

```
---  
masukkan ipk mahasiswa yang dicari:  
IPK: 3,7  
---  
menggunakan binary search  
---  
data mahasiswa dengan IPK : 3.7 ditemukan pada indeks 4  
nim     : 555  
nama    : anita  
kelas   : 2  
ipk     : 3.7
```

6.3.3. Pertanyaan

1. Tunjukkan pada kode program yang mana proses divide dijalankan!
2. Tunjukkan pada kode program yang mana proses conquer dijalankan!
3. Apa fungsi left, right, dan mid?
4. Jika data IPK yang dimasukkan tidakurut. Apakah program masih dapat berjalan? Mengapa demikian?
5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai
6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawaban :

1. $mid = (left + right) / 2;$
2. `return findBinarySearch(cari, left, mid-1);`
`return findBinarySearch(cari, mid+1, right);`
3. left berfungsi untuk batas indeks kiri (awal) dari area pencarian saat ini, right berfungsi sebagai batas indeks kanan (akhir) dari area pencarian saat ini, dan mid berfungsi sebagai indeks Tengah yang dihitung dari $(left + right) / 2$ yang digunakan untuk membandingkan nilai yang dicari
4. Program tetap bisa berjalan, tetapi hasilnya bisa saja salah, hal ini dikarenakan Binary search hanya berjalan benar pada data yang sudah terurut. Jika data tidak urut maka alur pencariannya (cari di kiri jika lebih kecil, di kanan jika lebih besar) menjadi tidak valid
5. Kode program yang digunakan adalah untuk data ascending, jika data yang diberikan descending maka kode program harus diubah seperti berikut :

```
int findBinarySearch(double cari, int left, int right) {
    int mid;
    if (right >= left) {
        mid = (left + right) / 2;
        if (cari == listMhs[mid].ipk) {
            return mid;
        } else if (listMhs[mid].ipk < cari) { // ubah > menjadi <
            return findBinarySearch(cari, left, mid - 1);
        } else {
            return findBinarySearch(cari, mid + 1, right);
        }
    }
    return -1;
}
```

6. Binary search menentukan data tidak ditemukan adalah ketika kondisi $right < left$ tercapai atau ketika fungsi mengembalikan -1. Ini maksudnya adalah area pencarian sudah tidak ada lagi sehingga data tidak ada dalam array
7. Perubahan :

- MahasiswaBerprestasi29
`public MahasiswaBerprestasi29(int n){`
`listmhs = new Mahasiswa29[n];`
`}`

- MahasiswaDemo29
`Scanner sc = new Scanner(System.in);`

`System.out.print("Masukkan jumlah mahasiswa: ");`
`int jumMhs = sc.nextInt();`
`sc.nextLine();`

`MahasiswaBerprestasi29 list = new MahasiswaBerprestasi29(jumMhs);`